集合-ArrayList

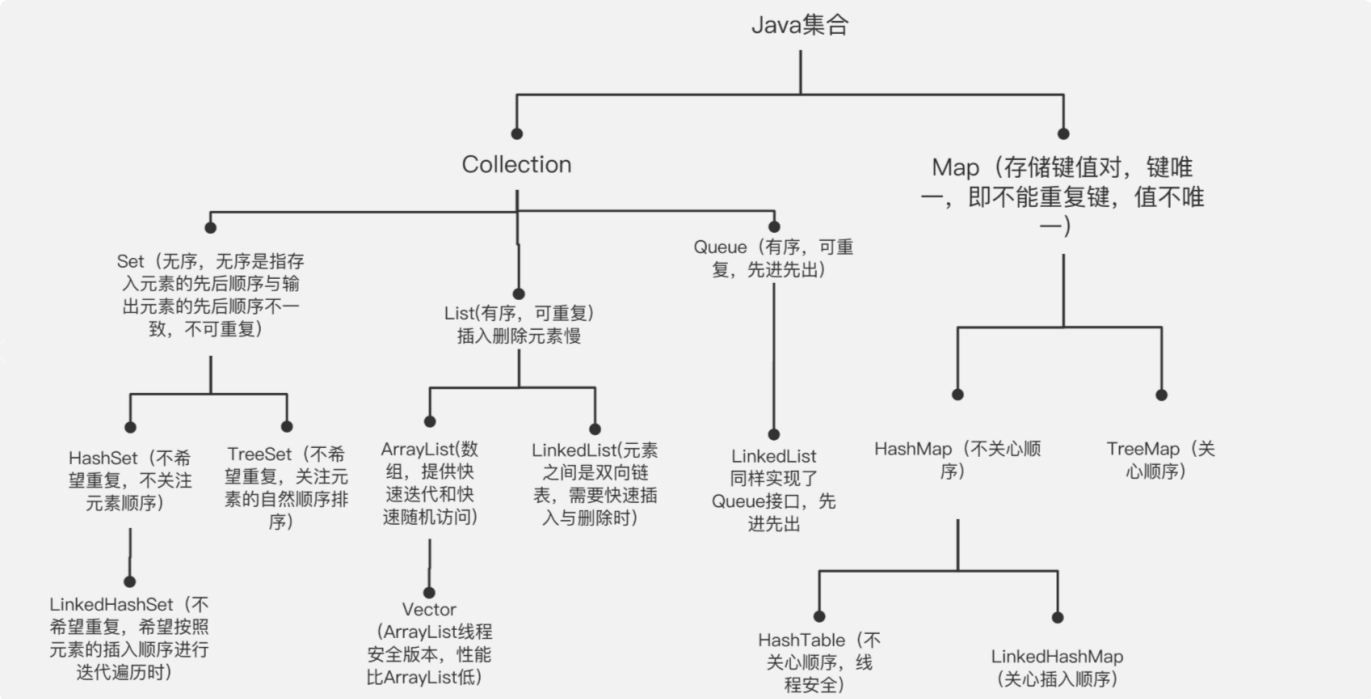
1. 集合整体图

2. 底层原理

3. 为什么ArrayList线程不安全

4. 解决方案

1. 集合整体图



线程安全：Vector、HashTable、StringBuffer、ConcurrentHashMap、Stack、Properties

线程不安全：其他的集合都是Set(HashSet、TreeSet)、List(ArrayList、LinkedList)、Map(HashMap、TreeMap)

2. 底层原理

1. 数据结构：ArrayList底层数据结构就是一个Objec[]数组，基于数组实现，可动态扩容
2. 初始容量：JDK8默认初始容量为10，最大容量为 Integer.MAX_VALUE – 8
3. 扩容机制：触发扩容的条件：当 size + 1 是否大于容量，如果大于容量则证明集合不够用

了，需要扩容。如果小与当前容量则证明集合还有容量不需要扩容！

4. 扩容的大小：默认oldCapacity + (oldCapacity >> 1) 即: 10+ (10>>1)，即：当前容量的 1.5 倍！

```
private void grow(int minCapacity) {
    // overflow-conscious code
    int oldCapacity = elementData.length;
    int newCapacity = oldCapacity + (oldCapacity >> 1);
    if (newCapacity - minCapacity < 0)
        newCapacity = minCapacity;
    if (newCapacity - MAX_ARRAY_SIZE > 0)
        newCapacity = hugeCapacity(minCapacity);
    // minCapacity is usually close to size, so this is a win:
    elementData = Arrays.copyOf(elementData, newCapacity);
}
```

3. 为什么ArrayList线程不安全

- **数据丢失和数组越界**

ArrayList 底层使用 Object[] 存储元素，通过 size 记录实际元素个数。当多个线程同时执行 add() 时，可能发生：

1.线程 A 和线程 B 同时读取到 size = 5，都认为数组还有空位，同时写入下标 5，导致其中一个线程覆盖另一个线程的数据，并且 size 最终只增加 1（丢失一个元素）。

2.多个线程同时触发扩容（grow()），导致数组引用被覆盖，部分线程使用过期的数组引用写入，抛出 ArrayIndexOutOfBoundsException。

- **modCount 与 size 不一致**

ArrayList 的 add() 方法会修改 modCount（用于快速失败机制）和 size。

并发环境下，这两个变量的修改不是原子的，可能导致集合内部状态损坏，甚至抛出 ConcurrentModificationException。

4. 解决方案

解决方案	适用场景	性能特点	注意事项
Vector	历史系统维护 JDK1.0兼容需求	全表锁性能差	已过时，不推荐

CopyOnWriteArrayList	读多写少场景	读操作无锁（O(1)） 写性能低（O(n)复制）	内存占用高 数据最终一致性
Collections.synchronizedList()	低并发兼容旧系统 简单迁移场景	全表锁（吞吐量低） 强一致性保证	遍历时需手动同步 `Iterator` 需额外加